

第五章 面向去中心化存储系统的可验证布尔查询处理机制

5.1 问题描述

5.1.1 系统架构

在分布式文件系统中，存在管理者节点、存储节点和客户端三种类型的节点。管理者节点作为协调者，主要负责元数据管理和存储调度，不直接存储文件数据。管理者节点维护文件到存储节点的映射关系，以提供基于文件标识符的数据检索服务。存储节点主要负责文件数据的物理存储和数据读写。用户文件在系统中被切分为固定大小的分片，这些分片被分布式地存放在不同物理机器的磁盘上。文件上传请求由客户端发起，随每个文件同步上传的还有一个关键词集合，用于为用户提供基于关键词的文件检索服务。为了实现上述服务，在“文件标识符-存储节点”映射之外，管理者节点需要再建立一个从关键词到文件标识符的映射。在系统为用户提供文件存储服务期间，客户端可以发起对文件关键词集合的修改请求，系统中相应映射将随之发生更新，从而为用户提供最新的文件检索结果。

安全模型：在分布式文件系统中，管理者节点被认为是诚实的节点，可以由

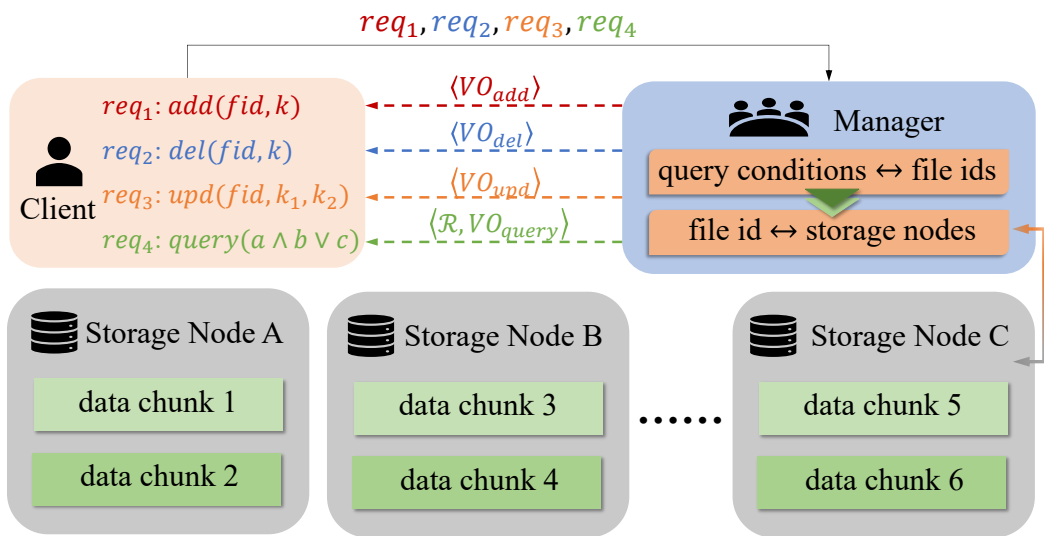


图 5.1: 分布式文件系统架构和可验证布尔查询示例

区块链智能合约实现。而链下的存储节点则被视为不可信的节点，由于硬件故障、黑客攻击或利益驱动，它们可能伪造、遗漏或提供过时的查询结果。因此，系统应该为返回给用户的查询结果提供有效性证明。一方面需要证明查询结果的正确性 (soundness)，即返回的所有文件确实满足查询条件。另一方面需要证明查询结果的完备性 (completeness)，即返回了系统中所有符合查询条件的文件。

5.1.2 问题定义

本文的分布式文件系统实现针对用户文件关键词的可验证增、删、改、查等操作的执行处理。其中，基于关键词的查询为复杂的布尔查询，且所有操作均需要系统提供有效性证明以通过客户端验证。上述操作的具体定义如下：

定义 5.1.1 (可验证文件关键词增加). 一个可验证文件关键词增加请求表示为 $add(fid, k)$ ，其中 fid 是一个文件标识符， k 是以 fid 为标识符的文件下一个待增加的关键词。分布式文件系统对该请求的响应表示为 $\langle VO_{add} \rangle$ ， VO_{add} 是系统在执行完成关键词增加后对该操作的有效性证明。

定义 5.1.2 (可验证文件关键词删除). 一个可验证文件关键词删除请求表示为 $del(fid, k)$ ，其中 fid 是一个文件标识符， k 是以 fid 为标识符的文件下一个待删除的关键词。分布式文件系统对该请求的响应表示为 $\langle VO_{del} \rangle$ ， VO_{del} 是系统在执行完成关键词删除后对该操作的有效性证明。

定义 5.1.3 (可验证文件关键词更新). 一个可验证文件关键词更新的请求表示为 $upd(fid, k_1, k_2)$ ，其中 fid 是一个文件标识符， k_1 是以 fid 为标识符的文件下一个已有的关键词， k_2 是用于替换 k_1 的关键词。分布式文件系统对该请求的响应表示为 $\langle VO_{upd} \rangle$ ， VO_{upd} 是系统在执行完成关键词更新后对该操作的有效性证明。

定义 5.1.4 (可验证关键词布尔查询). 一个可验证关键词布尔查询的请求表示为 $query(q)$ ，其中 q 是由关键词 (如 a, b, c) 通过布尔逻辑算子 (如 $\wedge, \vee, \neg$) 组合而成的查询表达式 (例如 $q_1: a \wedge b$ 或 $q_2: a \wedge b \vee c$)。分布式文件系统对该查询请

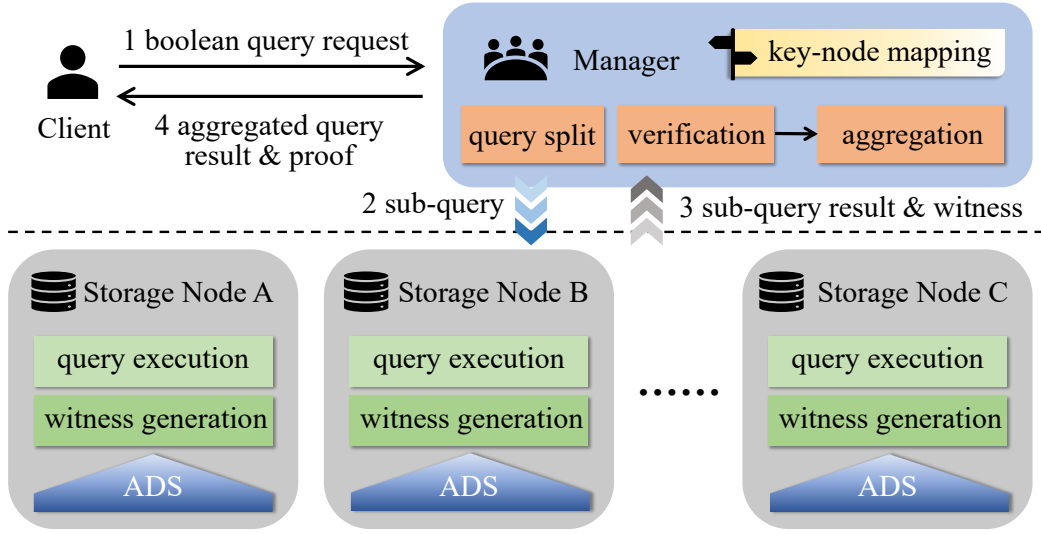


图 5.2: 分布式可验证布尔查询引擎概述

求的响应表示为 $\langle \mathcal{R}, VO_{query} \rangle$ ，其中 $\mathcal{R}$ 是系统返回的满足布尔表达式 q 的文件标识符集合， VO_{query} 是系统在执行完布尔查询后，为证明查询结果的正确性和完备性而提供的有效性证明。

5.2 分布式可验证布尔查询引擎

在分布式环境下实现复杂布尔查询的可验证性，既要求系统具备大规模扩展能力，又要在去中心化的信任模型下保证查询结果的真实性与完备性。本节将详细阐述分布式可验证布尔查询引擎的整体架构设计。该架构旨在通过解耦管理逻辑与存储执行，利用密码学累加器与前缀感知的路由机制，构建一个高性能且支持可验证复杂布尔逻辑的分布式查询系统。

5.2.1 查询引擎框架

分布式可验证布尔查询引擎采用“管理者-存储节点”双层拓扑架构，目标是在存储节点不可信任的分布式存储系统中支持形如 $\Phi = a \wedge b \vee c$ 的复杂布尔表达式查询，并向客户端提供常数级或对数级大小的密码学证明。

如图5.2所示，管理者节点维护一个“键-存储节点”映射，将键值对映射到各存储节点上进行分布式管理。各存储节点采用可验证结构（Authenticated Data

Structure, ADS) 对键值对进行索引, 并基于可验证结构进行查询任务的执行和证明生成。当客户端发起布尔查询请求时, 管理者节点将查询表达式拆分为多个单键的子查询, 并基于“键-存储节点”映射将子查询派发给相应的存储节点执行子查询处理。此后, 管理节点在收到存储节点返回的查询结果后对其进行有效性验证。在确认所有子查询结果有效后, 管理者将其聚合为最终查询结果, 并生成有效性证明。之后, 管理者节点将最终查询结果及其证明返回给客户端以响应客户端的查询请求。客户端验证查询证明以决定是否接收查询结果。此外, 为了保证中间查询结果的有效性可以被追溯, 管理者节点在验证其有效后将中间结果的证明发送至区块链上进行永久性保存。

具体而言, 查询引擎设计的关键在于“键-存储节点”映射、存储节点上的可验证结构、可聚合的查询处理与验证三个部分。

(1) 基于可扩展前缀哈希环的数据映射: 在分布式环境下实现高效查询, 首要挑战在于如何在海量节点间实现负载均衡的同时, 保持数据的语义局部性。传统的基于哈希的一致性哈希算法 (如 Chord) 虽然解决了节点的动态伸缩问题, 但其散列特性破坏了键的顺序性, 导致涉及前缀匹配或范围查询的布尔子任务需在全网广播, 产生巨大的通信冗余。为此, 本节设计了一种可扩展前缀哈希环, 通过提取键的十六进制前缀作为映射依据, 将全局键空间划分为基于前缀的连续区间, 并将其动态映射至哈希环上的虚拟存储节点。当系统发生节点加入或退出时, 仅需通过前缀分裂或合并即可完成数据重分布。可扩展前缀哈希环兼顾了随机哈希的均衡性与顺序索引的局部性, 使得具有相同前缀的子查询任务能够精确路由至特定节点集合, 显著降低了复杂布尔查询中的路由开销与查询延迟。本章在第5.2.2小节中详细介绍可扩展前缀哈希环。

(2) 存储节点内的累加器可验证字典树: 存储节点作为证明生成的原子单元, 面临着证明大小与验证效率之间的权衡。传统的基于前缀的可验证结构 Merkle Patricia Trie 在处理大规模数据集时, 其成员证明大小随数据量呈对数级增长, 且在证明“不存在” (即非成员证明) 和“未遗漏” (即完整性证明) 时逻辑较为复

杂，难以支撑高频并发的布尔原子操作。为此，本框架在存储节点内部构建了基于累加器的可验证字典树。该结构将密码学累加器嵌入到字典树的节点中。对于每一个子查询任务，存储节点通过累加器状态生成常数级大小的成员或非成员证明，以及查询结果的值完整性证明。累加器技术的引入消除了证明大小对树高的依赖，实现了常数级的证明生成与传输。这使得即使在底层数据量达到亿级规模时，存储节点返回给管理者节点的子查询结果证明依然保持极小的通信带宽占用。本章在第5.3节中详细介绍基于累加器的可验证字典树。

(3) 基于富语义累加器的查询处理与验证：在分布式布尔查询的最后阶段，管理者节点需要将来自多个存储节点的原子子查询结果集进行逻辑合并。由于布尔表达式在底层对应于集合的交并集运算，系统面临严峻的可验证性挑战。本框架引入了基于多项式表示的富语义累加器验证协议。该方案将集合运算转化为多项式代数约束，基于贝祖定理和集合的基数公式构造见证，并利用双线性映射验证交集和并集的有效性。对于布尔查询中复杂的嵌套逻辑，管理者节点采用递归证明合成技术，将多个碎片化见证合成为统一的全局有效性证明。该设计通过将复杂的集合逻辑映射为可验证的代数约束，实现了验证开销常数化与聚合逻辑透明化的深度融合，在显著降低客户端带宽负担的同时，从数学层面根除了分布式环境下管理者节点作恶的可能性。

5.2.2 可扩展前缀哈希环

为了实现根据字符串前缀的键值对和查询任务分发，本小节提出一种映射方案，称为可扩展前缀哈希环（Extendible Prefix Hashing Ring, EPRing）。EPRing是一种映射方式能够随数据量的增加而扩展的动态结构，它将冲突过重的表分散为多个轻量的表，在不需要重新哈希的情况下使负载尽可能均衡。

数据结构：与一致性哈希 [50, 51] 类似，EPRing 将存储节点和键的十六进制字符串映射到一个环上，每个存储节点对应环上一个范围的字符串。不同的是，EPRing 基于前缀对字符串进行映射。前缀长度（prefix length）标识了键中用于映

Index	0	1	2	3		15	16	17	18		31
Prefix	0	1	2	3		f	30	31	32		3f
Counter	5	7	9	100		8	4	5	6		7
SNID	SN ₀	SN ₁	SN ₂	16		SN _n	SN ₀	SN ₁	SN ₂		SN _n
ACC	A ₀ ¹	A ₁ ¹	A ₂ ¹	\		A _n ¹	A ₀ ²	A ₁ ²	A ₂ ²		A _n ²

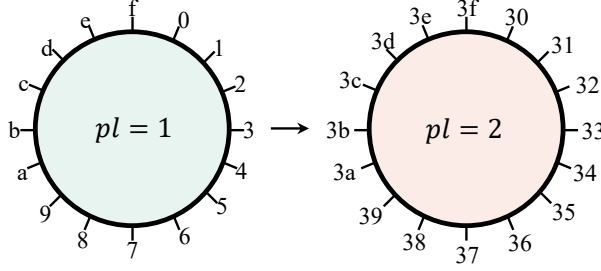


图 5.3: EPRing 示例

射的字符数量。初始前缀长度 pl 根据存储节点总数 n 确定，即将 pl 的初始值设定为满足 $16^{pl} \geq n$ 时的最小值。当某个前缀对应的键值对数量过多或访问频率过高时，例如超过某个阈值，该前缀可以分裂为一个新环。新环上的前缀比原环多一个字符，即新环的前缀长度为 $pl' = pl + 1$ 。存储节点以节点编号为序从前缀为 0 开始循环认领环上一个或多个前缀。若环上前缀为 pre 的地方发生分裂，则 EPRing 将新环上的前缀 $pre0$ 分配给原环 pre 对应的存储节点，新环上剩余的前缀则接着原环上存储节点的认领顺序继续分配。为了更加清晰地表述原环与新环的关系，原环被称为父环，新环则被称为原环的子环。

存储结构：EPRing 以变长数组的形式进行存储，数组的每个元素为一个四元组，包含前缀、计数器、存储节点编号和根节点摘要四个字段。计数器记录了当前前缀下的键值对总数（或它们被访问的总次数）。存储节点编号用于确定管理该前缀下所有键值对的存储节点。当计数器大于阈值后，该前缀将扩展一位，并在数组尾部添加十六个四元组，其中每个四元组的前缀在原前缀的基础上扩展一个十六进制字符。此时，原四元组的存储节点编号处将记录这十六个四元组的第 0 号四元组在数组中的索引号。这样做是便于在基于前缀进行键值对定位时能够快速跳转到相关的子环上。四元组中的根节点摘要是指相应存储节点上为该前缀下所有键值对构建的可验证结构的根节点摘要，其类型取决于存储节点上采用的可验证

结构类型。例如，若存储节点采用 MPT[20] 管理键值对，则管理者上四元组中的根节点摘要则为 MPT 的根节点哈希值。本文在存储节点上为键值对构建一个基于累加器的可验证字典树 AccTrie，因此四元组中的根节点摘要为 AccTrie 的根节点累加器值，本文将在第5.3节详细介绍 AccTrie。

5.2.3 可验证布尔查询处理

分布式可验证布尔查询处理的思路是将布尔表达式拆分为多个单键子查询，其中每个子查询的结果均为一个集合，最终查询结果由子查询结果集的交集并集运算得到。对查询结果的验证分为两个阶段：在第一个阶段，存储节点生成对子查询结果的证明，该证明由管理者节点验证，并将证明与验证结果上链以支持后续任意时刻对子查询结果的追溯；在第二个阶段，管理者节点生成对最终查询结果的有效性证明，即在不需要提供完整子查询结果的情况下证明它们运算结果的正确性，该证明由客户端进行验证。第一阶段子查询结果及其证明的生成由存储节点上的累加器可验证字典树实现，本文在第5.3节中详细阐述。本小节重点介绍第二阶段管理者节点对最终查询结果的计算及对其有效性证明的生成与验证。

假设布尔查询表达式为 $\Phi = a \wedge b \vee c$ ，若存储节点返回的子查询 $key = a, key = b, key = c$ 结果集分别为 A, B, C ，则最终查询结果集 $D = I \cup C$ ，其中 $I = A \cap B$ 。管理者节点先计算得到集合 I 并生成交集证明 $\pi_{A \cap B}$ 或 π_I ，然后计算最终查询结果 D 并生成并集证明 $\pi_{I \cup C}$ 或 π_D 。

本文采用密码学累加器 [12, 14, 52] 实现对交集和并集运算的证明。对于任意集合 S ，管理者节点可以计算其对应的多项式 $P_S(X)$ 和累加器值 Acc_S 。集合交并集运算的证明与验证则是基于集合多项式和累加器值进行。

交集的证明与验证：对于集合 A 和集合 B 的交集 I ，目标是使客户端在不需获取 A 和 B 具体内容的情况下通过密码学运算能够证明 I 的确是 A 和 B 的交集。其证明逻辑在于证明交集 I 对应的多项式 $P_I(X)$ 是 A, B 两个集合对应多项式 $P_A(X), P_B(X)$ 的最大公约数。进一步来说，需要证明 $P_I(X)$ 分别是 $P_A(X)$ 和

$P_B(X)$ 的公约数, 且 $P_I(X)$ 的两个商多项式 $Q_A(X)$ 和 $Q_B(X)$ 互质, 即保证交集的完备性。管理者节点通过提供一系列的见证值 (witness) 来让客户端可以通过配对运算检查是否满足上述两个条件。

为了证明 $P_I(X)$ 分别是 $P_A(X)$ 和 $P_B(X)$ 的公约数, 管理者节点计算出 $P_I(X)$ 的商多项式 $Q_A(X)$ 和 $Q_B(X)$, 并提供对它们的见证 W_{Q_A} 和 W_{Q_B} 。客户端使用以下双线性配对公式对上述条件进行验证:

$$e(Acc_A, g_2) \stackrel{?}{=} e(Acc_I, W_{Q_A}), e(Acc_B, g_2) \stackrel{?}{=} e(Acc_I, W_{Q_B}).$$

其中, g_2 是构造 W_{Q_A} 和 W_{Q_B} 的底数, 也是它们所在循环群 $\mathbb{G}_2$ 的生成元。

为了证明交集的完备性, 管理者节点基于贝祖定理证明 $P_I(X)$ 的两个商多项式 $Q_A(X)$ 和 $Q_B(X)$ 互质。根据贝祖定理可知, 若 $Q_A(X)$ 和 $Q_B(X)$ 互质, 则一定存在多项式 $A(X)$ 和 $B(X)$ 使得 $A(X)Q_A(X) + B(X)Q_B(X) = 1$ 。由此, 管理者节点找到 $A(X)$ 和 $B(X)$, 并生成对它们的见证 $W_A(X)$ 和 $W_B(X)$ 。客户端使用以下双线性配对公式对上述条件进行验证:

$$e(W_A, W_{Q_A}) \cdot e(W_B, W_{Q_B}) \stackrel{?}{=} e(g_1, g_2).$$

其中, g_1 是构造 W_A 和 W_B 的底数, 也是累加器值 Acc_A, Acc_B, Acc_I 和见证值 W_A, W_B 所在循环群 $\mathbb{G}_1$ 的生成元。

并集的证明与验证: 对于集合 A 和集合 B 的并集 U , 目标是使客户端在不需获取 A 和 B 具体内容的情况下通过密码学运算能够证明 U 的确是 A 和 B 的并集。并集的证明是基于集合的基数公式 $|A \cup B| = |A| + |B| - |A \cap B|$, 若集合 U 是 A 和 B 的并集, 它们的多项式一定满足等式:

$$P_A(X) \cdot P_B(X) = P_U(X) \cdot P_{A \cap B}(X).$$

并集的证明包含交集的累加器值 Acc_I, Acc_U 和交集证明, 即累加器值 Acc_A, Acc_B

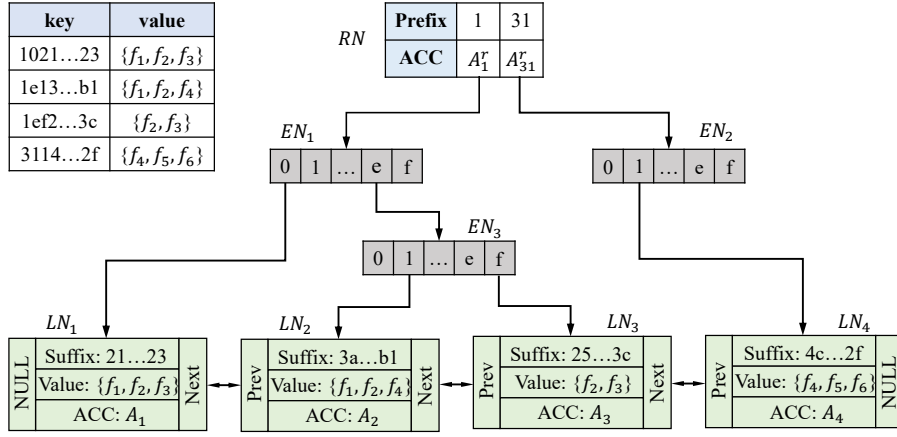


图 5.4: AccTrie 示例

和见证值 $W_{Q_A}, W_{Q_B}, W_A, W_B$ 。客户端首先通过交集证明验证交集累加器值的有效性，然后再验证以下等式是否成立：

$$Acc_A + Acc_B \stackrel{?}{=} Acc_U + Acc_I.$$

若等式成立，则证明并集有效；否则，并集无效。

基于密码学累加器，布尔查询处理方案将复杂的集合运算问题转化为多项式代数问题，再通过双线性配对等密码学工具，实现了客户端在不需子查询结果集合具体内容的情况下，即可进行可靠的查询结果有效性验证。

5.3 基于累加器的可验证字典树

管理者节点将“关键词-文件编号”键值对映射到各存储节点上。存储节点为其收到的所有键值对构建一个可验证结构以支持基于关键词的可验证查询。本文提出一个基于累加器的可验证字典树 AccTrie。本节首先介绍 AccTrie 的数据结构，然后详细介绍 AccTrie 为键值对生成证明与验证证明的算法，最后给出一个面向数据迁移的 AccTrie 持久化方案。

5.3.1 数据结构

AccTrie 是一种将字典树 (Trie Tree) 和密码学累加器 (Cryptographic Accumulator) 相结合以支持关键词可验证查询处理的可验证结构。字典树提供了基于键检索值的逻辑路径, 具有良好的前缀匹配和有序性。密码学累加器将集合压缩为一个摘要值, 能够为集合元素的存在性以及集合运算结果提供有效性证明。

节点类型: AccTrie 将键值对的键 (即关键词) 编码为 16 进制哈希码, 并采用树结构对键值对进行索引。具体来说, AccTrie 包含三类节点, 分别是叶子节点、扩展节点和根节点。各类节点包含的内容如下。

- **叶子节点:** 该类节点保存“关键词-文件编号”键值对、一个前向指针、一个后向指针和一个累加器值。对于键, 叶子节点仅记录它的后缀。值是一个文件编号集合, 所有包含该关键词的文件所对应的编号将被包含在这个集合中。将所有叶子节点放在同一层可以组成一个按照键的字典序排列的节点序列。每个叶子节点的前向指针指向上述序列中它左边的叶子节点, 后向指针指向它右边的叶子节点。叶子节点中的累加器值包含文件编码集合中的每个元素、元素个数、当前叶子节点和其前序叶子节点对应的键。
- **扩展节点:** 该类节点保存一个长度为十六的指针数组, 每个指针指向一个孩子节点。扩展节点的孩子节点为一个扩展节点或一个叶子节点。
- **根节点:** 该类节点包含一个三元组列表, 其中每个三元组包含前缀、累加器值和子指针三个字段。其中子指针指向一个扩展节点或一个叶子节点, 前缀是其所有孩子节点中键的公共前缀, 累加器值包含其孩子节点中各叶子节点上的累加器值。

5.3.2 证明生成与验证

AccTrie 通过在 Trie 树的叶子节点和根节点中嵌入累加器, 实现了对键值对的存在性证明。同时, 通过在叶子节点的累加器值中包含前一个叶子节点对应的

键，将所有叶子节点串连起来，能够为不在索引中的键提供不存在证明。接下来将详细介绍存在性证明和不存在证明的生成与验证。

存在性证明的生成与验证：一个键值对 (k, fid) 在 $AccTrie$ 中的存在性证明包括 k 所在的叶子节点累加器值 $LN.Acc$ 和 fid 在该累加器中的成员证明 π_1 ，以及根节点中相应三元组的累加器值 $RN.Acc$ 和 $LN.Acc$ 在 $RN.Acc$ 中的成员证明 π_2 。存在性证明的验证包括两个累加器成员证明的验证，即验证 π_1 以证明 fid 在 $LN.Acc$ 中存在和验证 π_2 以证明 $LN.Acc$ 的有效性。

不存在证明的生成与验证：要证明一个键 k 在 $AccTrie$ 中不存在，需要先根据前缀访问到与 k 最相近的两个键 k_p 和 k_n 对应的叶子节点，其中 k_p, k, k_n 满足字典序。然后通过证明两个叶子节点的连续性来证明 k 不存在。具体来说，从根节点开始进行前缀匹配，直到找到一个扩展节点上与 k 匹配的索引号对应的孩子指针不存在。接下来分别访问与该索引号最近的左右两个非空孩子节点（记为 lc 和 rc ）以找到完整的 k_p 和 k_n 及其对应的叶子节点。 k_p 的后缀为访问以 lc 为祖先节点的子树时每层最右边非空孩子节点组成的路径， k_n 的后缀则为访问以 rc 为祖先节点的子树时每层最左边非空孩子节点组成的路径。

最终， k 的不存在证明包含三个键 k_p, k, k_n 对应叶子节点的累加器值 $LN_n.Acc$ 及 k_p 和 k_n 在该累加器中的成员证明 π_1 和 π_2 、根节点中相应三元组的累加器值 $RN.Acc$ 及 $LN_n.Acc$ 在该累加器中的成员证明 π_3 。不存在证明的验证过程包括以下步骤：（1）验证 k_p, k, k_n 满足字典序；（2）验证 π_1 以证明 k_p 有效且其对应的叶子节点 LN_p 与 k_n 对应的叶子节点 LN_n 相连；（3）验证 π_2 以证明 k_n 有效；（4）验证 π_3 以证明 $LN_n.Acc$ 的有效性。

在基于关键词的文件检索场景中，一个键 k 可能对应多个文件编号，例如 $fid_1, fid_2, \dots, fid_n$ ，其中每个文件编号 fid_i 来自于一个键值对 (k, fid_i) 。 $AccTrie$ 将所有文件编号放入一个集合中，并将每个叶子节点中值集合的元素个数计入累加器中，从而能够为查询结果提供完整性证明。下面给出值完整性证明的生成与验证。

值完整性证明的生成与验证：键值对 $(k, \{fid_1, \dots, fid_n\})$ 的值完整性证明包括

相应叶子节点的累加器值 $LN.Acc$ 、值集合中元素个数 n 在该累加器中的成员证明 π_1 和根节点中相应三元组的累加器值 $RN.Acc$ 及 $LN.Acc$ 在该累加器中的成员证明 π_2 。与存在性证明类似，值完整性证明的验证也包括两个累加器成员证明的验证，即验证 π_1 以证明元素个数 n 的有效性和验证 π_2 以证明 $LN.Acc$ 的有效性。

针对客户端对文件关键词集合的增、删、改、查等请求，存储节点基于 **AccTrie** 的上述证明对其执行结果给出有效性证明。具体来说，当用户对标识符为 fid 的文件增加一个关键词 k 时，存储节点在其 **AccTrie** 中增加一个键值对 (k, fid) 并生成 (k, fid) 的存在性证明。当用户对标识符为 fid 的文件删除一个关键词 k 时，存储节点在其 **AccTrie** 中删除一个键值对 (k, fid) 并生成 (k, fid) 的不存在性证明。当用户对标识符为 fid 的文件将一个关键词 k_1 改为 k_2 时，相关存储节点在其 **AccTrie** 中删除 (k_1, fid) 并生成 (k_1, fid) 的不存在性证明，同时相关存储节点在其 **AccTrie** 中增加 (k_2, fid) 并生成 (k_2, fid) 的存在性证明。当用户基于关键词 k 进行查询时，存储节点将返回其对应的文件标识符集 $\{fid_1, fid_2, \dots\}$ ，并生成对键值对 $(k, \{fid_1, fid_2, \dots\})$ 的存在性证明和值完整性证明。

5.3.3 持久化存储

在存储节点上，**AccTrie** 的持久化需要考虑到数据在存储节点间的再分配和索引结构的变动，需要实现现在兼顾 **AccTrie** 动态性的同时使读写代价尽可能低。由第5.2节可知，可扩展哈希环 **EPRing** 会根据前缀下键值对的数量动态调整键值对在各存储节点上的分布，若某个前缀下键值对数量过多时，它们将被分为 16 份并重新分配到不同的存储节点上以均衡各存储节点的服务压力。在此期间，**AccTrie** 将被切割，其数据将被转移至其他存储节点。因此，**AccTrie** 的持久化需要考虑到其后续的数据迁移，尽可能地将相关数据存放在一起。此外，**AccTrie** 中键值对的增删改等操作会导致扩展节点分裂或出现新的叶子节点。因此，**AccTrie** 的持久化还需要能够灵活应对索引结构的变动。为此，本文提出基于前缀的分页存储方案，并为 **AccTrie** 设计了页面内的数据组织结构。在此方案下，数据迁移时可一次性读

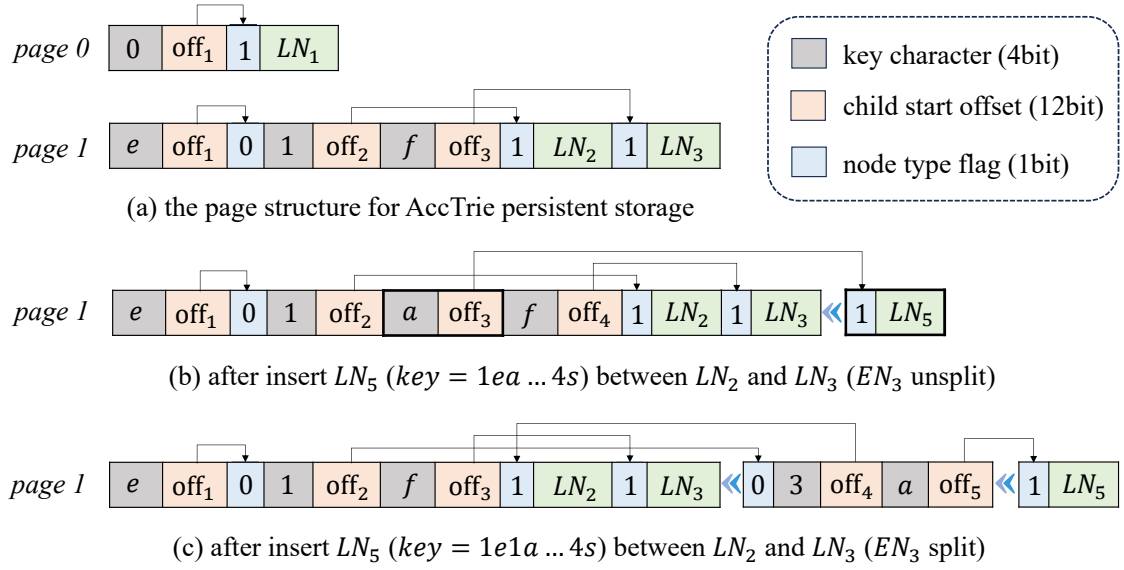


图 5.5: AccTrie 的持久化存储页面组织结构示例

取并传输整页数据，产生的 I/O 代价非常低，而且实现了单个页面内的数据几乎以 append-only 的方式写入，降低了索引节点创建和分裂造成的数据重写开销。

组织结构：为了便于数据迁移，持久化存储方案将 AccTrie 基于前缀进行分页存储。具体来说，对于 AccTrie 根节点中的每个三元组，若其孩子指针指向一个叶子节点，则将该叶子节点单独存在一个页面中。若其孩子指针指向一个扩展节点，则将该扩展节点的每个孩子分支分别存在一个页面中。以图5.4中的 AccTrie 为例，对于前缀为 1 的三元组，其孩子指针指向扩展节点 EN_1 ，且 EN_1 下存在以 0 和 e 为前缀的两个孩子分支。在持久化该 AccTrie 时，分页存储方案会将叶子节点 LN_1 放入页面 $page\ 0$ ，将扩展节点 EN_3 和叶子节点 LN_2, LN_3 放入 $page\ 1$ 。

在单个页面内，排在最前面的四个比特为当前分支的公共前缀字符，此后扩展节点被持久化为由键字符和孩子节点起始偏移量组成的数组，叶子节点是被序列化处理后的字符串。每个扩展节点由三个类型的字段组成，分别是节点类型标识位、键字符和子节点起始偏移量。每个叶子节点包含节点类型标识位和节点序列化字符串两个字段。需要说明的是，节点序列化字符串的长度作为该字符串的一部分被放在节点类型标识位之后，长度一般为八个比特。节点类型标识位用于区分节点类型，长度为一个比特。节点类型标识位为 0 时表示接下来的数据是扩

展节点，为 1 时表示接下来的数据是叶子节点。扩展节点的键字符是 0 到 f 中的单个字符，占用四个比特，每个键字符之后紧接着与其对应的子节点起始偏移量。在 4KB 的页面中子节点起始偏移量大小为十二个比特。若某个键字符对应的子节点为空时，其子节点起始偏移量记为 0。

例 5.3.1. 图 5.5 展示了图 5.4 所示 *AccTrie* 中前缀为 1 的子树持久化后的页面组织结构。扩展节点 EN_1 中以 0 和 1 为前缀的两个分支分别被持久化为 *page 0* 和 *page 1*。在 *page 0* 中，前 4 个比特为公共前缀字符 0000，之后是 LN_1 节点类型标识位的页内偏移量。在 *page 1* 中，前 4 个比特为公共前缀字符 1110 (即十六进制字符 e)，之后是 EN_3 的十六个键字符及其子节点偏移量。注意，图中只展示非空子节点，包括 1 和 f ，非空子节点未画出。键字符 1 的子节点偏移量 off_2 是叶子节点 LN_2 节点类型标识位的页内偏移量。同样地，键字符 f 的子节点偏移量 off_3 是叶子节点 LN_3 节点类型标识位的页内偏移量。

数据插入：当有新的键值对插入 *AccTrie* 时可能会引发 *AccTrie* 结构的变化，此时相应的页面内的数据组织结构也将作出相应调整。若键值对的插入生成了新的叶子节点，*AccTrie* 结构的变化将存在两种情况：一种是叶子节点直接是某个已存在扩展节点的子节点，另一种是在某个扩展节点发生分裂后，该叶子节点是新扩展节点的子节点。在第一种情况下，新叶子节点将被直接添加到页面数据尾部，并将相应扩展节点的相关键字符后的子节点起始偏移量设置为新叶子节点节点类型标识位的页内偏移量。如图 5.5(b) 所示，当插入一个键为 $1ea...4s$ 的叶子节点 LN_5 时，该叶子节点序列化后的字符串被直接添加在 *page 1* 中 LN_3 之后，节点类型标识位为 1。从键字符 e 开始，由 off_1 找到扩展节点 EN_3 的起始位置，随后将键字符 a 后的 off_2 设置为 LN_5 节点类型标识位的页内偏移量。

若新叶子节点的插入触发了新扩展节点的生成，则页面内存入新扩展节点和新叶子节点。新扩展节点以为 0 的节点类型标识位开始，包含十六个键字符及其子节点起始偏移量。原扩展节点相应键字符后的偏移量被设置为新扩展节点节点类型标识位的页内偏移量，同时原扩展节点该键字符对应的子节点将作为新扩展

节点的一个子节点，相应键字符后的子节点起始偏移量将被设置为该子节点节点类型标识位的页内偏移量。随后，新叶子节点被序列化后存在新扩展节点之后，并将新扩展节点相应键字符后的子节点起始偏移量设置为新叶子节点节点类型标识位的页内偏移量。如图5.5所示，当插入的叶子节点键为 $1e1a...4s$ 时，在扩展节点 EN_3 的键字符 1 处已存在叶子节点 LN_2 。因此， EN_3 分裂生成新的扩展节点 EN_4 ， EN_3 中键字符 1 后的子节点起始偏移量 off_2 被设置为 EN_4 节点类型标识位的页内偏移量。 EN_4 中键字符 3 后的子节点起始偏移量 off_4 和 off_5 分别被设置为叶子节点 LN_2 和 LN_5 节点类型标识位的页内偏移量。

数据迁移：当 EPRing 监测到某个前缀下的键值对数量过多时，它为该前缀扩展一位字符从而将其键值对集合分裂为十六个子集，并将它们重新分配到不同的存储节点上。随之发生的则是存储节点上 AccTrie 的拆分与数据迁移。如前面介绍的分页存储方案所述，AccTrie 按照其第一个扩展节点的键字符将不同孩子分支划分存储在不同的页面中。基于此，存储节点在接收到 EPRing 扩展后的再分配指令后，可以直接将各个页面发送给相应的目标存储节点。目标存储节点在收到一个页面后，会根据页内的第一个扩展节点将其划分为多个页面，依然保持分页存储。

为了保证迁移后数据的有效性，目标存储节点会将所接收页面中所有叶子节点的累加器值累加到一个累加器中，然后将该累加器值发送给管理者节点。管理者节点在接收到所有累加器值后将其合并为一个累加器值，并与本地保存的原累加器值对比，两者一致则将各累加器值记录到 EPRing 的四元组中，数据迁移完成。此外，为了在数据迁移期间确保系统仍能提供服务，原存储节点暂不删除待迁移的页面并为客户端提供查询服务。键值对的增、删、改等请求则由管理者节点暂时将其分配给相关目标节点，在数据迁移完成后由目标节点一并执行。原存储节点上暂存的页面也将在数据迁移完成后被删除。